МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений.

Студент гр. 4343 _____ Селезнев Д. Е.

Преподаватель _____ Пуеров Г. Ю.

Санкт-Петербург

2026

Цель работы.

Изучение и практическое применение итерационных методов приближенного решения нелинейных уравнений – метода бисекции, метода хорд, метода Ньютона и метода простых итераций – с использованием программных функций на языке C++, а также исследование влияния точности исходных данных на процесс вычислений.

Задание.

Функция варианта №17: $f(x) = e^x - \arccos(\sqrt{x})$

Метод бисекции:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т. е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).
2) Составить подпрограмму вычисления функции $f(x)$.
3) Составить головную программу, содержащую обращение к подпрограмме f(x), BISECT, Round и индикацию результатов.
4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps.
5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Delta.

Метод хорд:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т. е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям применимости метода).
2) Составить подпрограмму - функцию вычисления функции $f(x)$, предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.

3) Составить головную программу, вычисляющую корень уравнения $f(x)=0$ и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.
4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Метод Ньютона:

1) Графически или аналитически отделить корень уравнения $f(x)=0$ (т. е. найти отрезки [Left, Right], на котором функция $f(x)$ удовлетворяет условиям сходимости метода Ньютона).
2) Составить подпрограммы - функции вычисления $f(x)$, $f'(x)$, предусмотрев округление их значений с заданной точностью Delta.
3) Составить головную программу, вычисляющую корень уравнения $f(x)=0$ и содержащую обращение к подпрограммам $f(x)$, f'(x), Round, NEWTON и индикацию результатов.
4) Выбрать начальное приближение корня x_0 из [Left, Right] так, чтобы $f(x)\ f''(x)>0$.
5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Метод простых итераций:

1) Графически или аналитически отделить корень уравнения $f(x)=0$.
2) Преобразовать уравнение $f(x)=0$ к виду $x=\varphi(x)$ так, чтобы в некоторой окрестности [Left, Right] корня производная $\varphi'(x)$ удовлетворяла условию $|\varphi'(x)|\le q<1$. При этом следует иметь в виду, что чем меньше величина q, тем быстрее последовательные приближения сходятся к корню.
3) Выбрать начальное приближение, лежащее на [Left, Right].

4) Составить подпрограмму для вычисления значений $\varphi(x)$, $\varphi'(x)$, предусмотрев округление вычисленных значений с точностью Delta.
5) Составить головную программу, вычисляющую корень уравнения и содержащую обращение к программам $\varphi(x)$, $\varphi'(x)$ и ITER и индикацию результатов.
6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Выполнение работы.

1. Отделение (изоляция) корня

Функция содержит:

- e^x – определена при всех x
- $\sqrt{x}$ – нужно $x \geq 0$
- $arc\cos(t)$ – аргумент должен быть в диапазоне [-1, 1]

Таким образом область определения: $x \in [0,1]$.

Проверим значения функции на концах:

При $x = 0$: $f(0) = e^0 - \arccos(0) = 1 - \frac{\pi}{2} \approx 1 - 1.5708 = -0.5708$

Получается $f(0) < 0$.

При $x = 1$: $f(1) = e^1 - \arccos(1) = e - 0 \approx 2.718$

Получается $f(1) > 0$.

Функция непрерывна на $[0,1]$ и значения на концах имеют разные знаки, следовательно на отрезке $[0,1]$ существует хотя бы один корень.

Так как e^x – строго возрастает, $arc\cos(\sqrt{x})$ – строго убывает, тогда их разность строго возрастает. Следовательно, корень единственный.

График функции см. на рисунке 1.

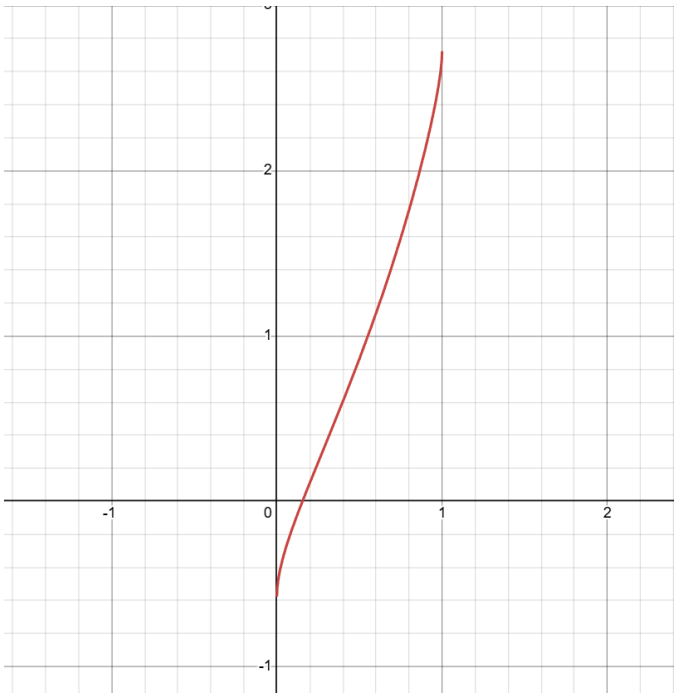

Рисунок 1 – График функции варианта №17

1. Подпрограмма Round

Функция предназначена для округления значения x до ближайшего кратного величине *Delta*. Перед выполнением проверяется корректность параметра *Delta*: если $\Delta \leq 10^{-9}$, работа программы завершается с ошибкой. Округление выполняется отдельно для положительных и отрицательных значений x, что обеспечивает корректный результат. Данная функция используется для моделирования погрешности вычислений.

2. Подпрограмма вычисления функции $\boldsymbol{f(x)}$

Функция предназначена для вычисления значения выражения $e^x - \arccos(\sqrt{x})$ с возможным моделированием округления. Сначала она проверяет, что входное значение x находится в допустимом диапазоне [0,1]. Если x меньше 0 или больше 1, функция возвращает *NAN* (не число), чтобы сигнализировать о недопустимом входе.

Затем вычисляется само выражение *exp(x) - acos(sqrt(x))*. Здесь *exp(x)* – это экспонента числа x, *sqrt(x)* — квадратный корень из x; а *acos(sqrt(x))* – арккосинус от корня из x в радианах.

После этого функция проверяет, задан ли параметр *Delta*, который задаёт величину округления. Если $Delta > 0$, значение округляется до ближайшего кратного *Delta* с помощью функции *Round*, имитируя некоторую погрешность измерения или вычисления. Если $Delta \leq 0$, функция возвращает точное вычисленное значение без округления.

3. Подпрограмма вычисления производной $\boldsymbol{F'(x)}$

Функция предназначена для вычисления производной $F'(x) = e^x + \frac{1}{2\sqrt{x}\sqrt{1-x}}$. Область допустимых значений здесь строже, чем у исходной функции. Поскольку в формуле присутствуют выражения $\sqrt{x}$ и $\sqrt{1-x}$ в знаменателе, значение xдолжно принадлежать интервалу (0, 1). Если $x \leq 0$ или $x \geq 1$, функция возвращает *NAN*, так как производная в этих точках не определена (знаменатель обращается в ноль). Далее вычисляется само выражение. Аналогично функции F(x), если параметр *Delta* > 0, результат

округляется с помощью функции *Round*, что позволяет моделировать влияние погрешности вычислений. Если $Delta \leq 0$, возвращается точное значение производной.

4. Подпрограмма вычисления второй производной $F''(x)$

Ф Функция предназначена для вычисления второй производной $F''(x) = e^x - \frac{1-2x}{4x^{\frac{3}{2}}(1-x)^{3/2}}$. Область допустимых значений определяется наличием выражений $x^{3/2}$и $(1-x)^{3/2}$в знаменателе, поэтому значение x должно принадлежать интервалу (0, 1). При $x \leq 0$ или $x \geq 1$ функция возвращает *NAN*. Вычисление производится с использованием промежуточных значений, что упрощает реализацию выражения. При необходимости моделирования погрешности результат округляется с точностью *Delta* с помощью функции *Round*, в противном случае возвращается точное значение.

5. Численные методы

5.1. Метод бисекции

В методе берется интервал [a, b], где функция меняет знак ($f(a) \cdot f(b) < 0$), и делит его пополам. Вычисляется середина $x = (a + b) / 2$, и проверяется знак *f(x)*. Если $f(x) \cdot f(a) < 0$, значит корень находится в левой половине [a, x], и правая граница сдвигается: b = x. Иначе корень в правой половине [x, b], и левая граница сдвигается: a = x. Этот процесс повторяется, пока длина интервала не станет меньше удвоенной точности: $(b - a) < 2 \cdot \varepsilon$. В коде это реализовано через цикл *while ((Right - Left) >= E)* где *E = 2-Eps*.

5.2. Метод хорд

Метод проводит прямую линию(хорду) через точки (a, f(a)) и (b, f(b)) и находит точку пересечения этой прямой с осью *X* по формуле: $x = a - (b - a) \cdot f(a) / (f(b) - f(a))$. Эта точка обычно ближе к корню, чем середина отрезка, потому что

учитывается наклон функции. После вычисления x проверяется знак f(x): если $f(x) \cdot f(a) < 0$, то корень в [a, x] и сдвигается правая граница (b = x), иначе сдвигается левая (a = x). Цикл продолжается до тех пор, пока $|f(x)| \geq Eps$, то есть пока значение функции не станет достаточно близким к нулю. Условие завершения *while (fabs(Y) >= Eps)* проверяет именно это.

5.3. Метод Ньютона

- **Выбор начального приближения**

  В качестве начального приближения необходимо выбрать точку, удовлетворяющую условию $F(x_0) \cdot F''(x_0) > 0$. Так как на интервале (0, 1) выполняется $F''(x) > 0$, достаточно выбрать точку, в которой $F(x_0) > 0$, то есть правее корня. В качестве начального приближения выбрано значение, поскольку оно удовлетворяет условию сходимости и находится ближе к корню по сравнению с другими допустимыми значениями (например, 0.5 или 0.7), что обеспечивает более быструю сходимость метода.

- **Реализация**

  Метод реализует итерационный процесс с использованием первой производной функции. Производная используется для определения наклона касательной к графику функции в текущей точке, что позволяет более точно приблизиться к корню. На каждой итерации вычисляются значения $F(x)$ и $F1(x)$, после чего определяется шаг $DX = Y / Y1$ и новое приближение $X = X - DX$. Перед началом итераций на отрезке [*Left*, *Right*] с малым шагом вычисляются оценки и $M_2 = \max | F2(x) |$, которые используются для определения уточнённой точности $Eps0 = 2 \cdot m1 \cdot Eps/M2$. Итерационный процесс завершается либо при выполнении условия $| X - XPrev |< Eps0$, либо при выполнении основного критерия

$|DX| < Eps$. В процессе работы предусмотрена проверка обращения производной в ноль. Число итераций учитывается в переменной N.

5.4. Метод простых итераций

- **Выбор функции итерации $\varphi(x)$ (метод релаксации)**

  При выборе функции $\varphi(x)$ используется метод релаксации, согласно которому исходное уравнение приводится к виду $x = \varphi(x) = x - \lambda F(x)$, где λ— параметр релаксации. Для обеспечения сходимости метода необходимо выполнение условия $|\varphi'(x)| < 1$, при этом $\varphi'(x) = 1 - \lambda F'(x)$, откуда следует $0 < \lambda F'(x) < 2$ и, следовательно $\lambda < \frac{2}{\max F'(x)}$. По результатам анализа производной на интервале (0, 1) было вычислено $F'_{\max} \approx 7.7$, поэтому $\lambda < \frac{2}{7.7} \approx 0.26$. Значения λ, близкие к верхней границе, приводят к увеличению коэффициента сжатия q и могут вызывать неустойчивость итерационного процесса, поэтому параметр выбирается с запасом. Для анализа рассмотрим зависимость $q = max\,|1 - \lambda F'(x)|$:

λ	$q = max\,\|1 - \lambda F'(x)\|$	Сходимость
0.10	0.73	Медленная
0.15	0.60	Устойчивая
0.20	0.54	Устойчивая
0.25	0.93	Близко к границе
0.26	1.00	Потеря сходимости

  Из таблицы видно, что при увеличении λ коэффициент сжатия сначала уменьшается, затем начинает расти и достигает 1, что соответствует границе. Таким образом, выбирается значение $\lambda = 0.2$, так как оно находится на безопасном расстоянии от критического значения и обеспечивает устойчивую и достаточно быструю сходимость метода.

- **Реализация**

Метод реализует итерационный процесс вида $x_{n+1} = \varphi(x_n)$. На каждой итерации вычисляется новое значение через функцию $\varphi(x)$, после чего оно становится следующим приближением. Процесс продолжается до тех пор, пока разность между соседними значениями не станет меньше заданной точности Eps.

6. Головная программа

Программа использует численные методы для нахождения корня функции на отрезке [0,1]. Для каждого метода проведены два эксперимента, соответствующих требованиям задания.

6.1. Исследование зависимости числа итераций от точности Eps

В первом эксперименте вычисляется корень функции для различных значений точности *Eps*.

Для каждого значения точности:

- вызывается соответствующая функция (*BISECT*, *HORDA, NEWTON, ITER*);
- фиксируется найденный корень и число итераций, потребовавшихся для сходимости;
- результаты выводятся в табличном виде.

Данные позволяют построить график зависимости числа итераций от *Eps*.

6.2. Исследование чувствительности метода к ошибкам (Delta)

Во втором эксперименте моделируются ошибки исходных данных с помощью глобальной переменной *Delta*, которая задаёт точность округления значений функции через функцию *Round*. Используются значения *Delta* = 0.1, 0.01, 0.001, 0.0001. При фиксированной точности $Eps = 10^{-6}$ вычисляется корень функции и число итераций, а результаты выводятся в виде таблицы, что позволяет оценить влияние погрешностей исходных данных на работу метода. После

эксперимента переменная *Delta* обнуляется, возвращая работу метода к стандартной точности.

Исследование.

Анализ результатов зависимости числа итераций от точности *Eps* показал, что с уменьшением значения *Eps* число итераций возрастает для всех методов.

Метод бисекции:

Для метода бисекции это полностью соответствует теоретической оценке: чем строже требуемая точность, тем больше шагов требуется для сходимости. Метод стабильно находит корень, близкий к реальному значению функции на заданном отрезке.

Метод хорд:

Для метода хорд наблюдается более быстрая сходимость: для тех же значений *Eps* требуется меньшее число итераций. Метод имеет сверхлинейную скорость сходимости. Это объясняется тем, что на каждой итерации используется пересечение хорды (линии, соединяющей точки на графике функции) с осью X в качестве нового приближения к корню. Если функция гладкая и почти линейна на выбранном интервале, новое приближение обычно ближе к корню, чем середина отрезка, что уменьшает число итераций по сравнению с линейной скоростью метода бисекции.

Метод Ньютона:

Для метода Ньютона наблюдается ещё более быстрая сходимость по сравнению с методами бисекции и хорд. При приближении к корню метод демонстрирует квадратичную скорость сходимости: число верных цифр удваивается с каждой итерацией, если начальное приближение достаточно близко к корню и функция имеет непрерывную первую и вторую производные на рассматриваемом участке. Однако метод чувствителен к выбору начального приближения: при плохом выборе стартовой точки может не сойтись или выйти за пределы интервала. Если функция имеет горизонтальную касательную или критическую точку рядом с корнем, это также может

замедлить сходимость. В идеальных условиях метод Ньютона требует значительно меньше итераций, чем метод хорд, особенно при высокой точности Eps.

Метод простых итераций:

Метод простых итераций демонстрирует более медленную сходимость по сравнению с методом Ньютона и методом хорд. Скорость сходимости является линейной и существенно зависит от выбора итерационной функции $\varphi(x)$. Для сходимости необходимо выполнение условия сжатия ($|\varphi'(x)| < 1$ в окрестности корня). В отличие от метода бисекции с фиксированной скоростью сходимости, здесь она зависит от коэффициента сжатия $q = max|\varphi'(x)|$: при $q < 0.5$ метод может сходиться быстрее бисекции, при $q > 0.5$ – медленнее. В рассматриваемом случае $q \approx 0.54$, поэтому метод показывает сходимость, близкую к бисекции. При удачном выборе функции преобразования метод может показывать удовлетворительную скорость, однако в общем случае требует большего числа итераций при уменьшении Eps.

Графики зависимости числа итераций от Eps представлены на рисунках 2, 3, 4, 5 и 6.

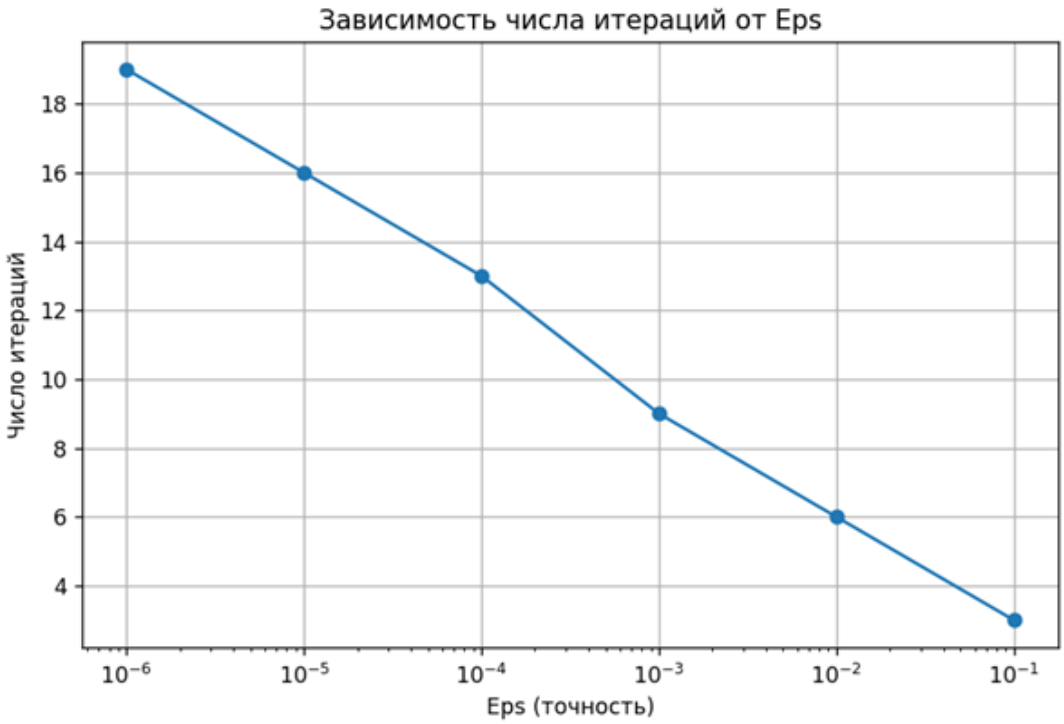

Рисунок 2 - Зависимости числа итераций от точности Eps в методе бисекции

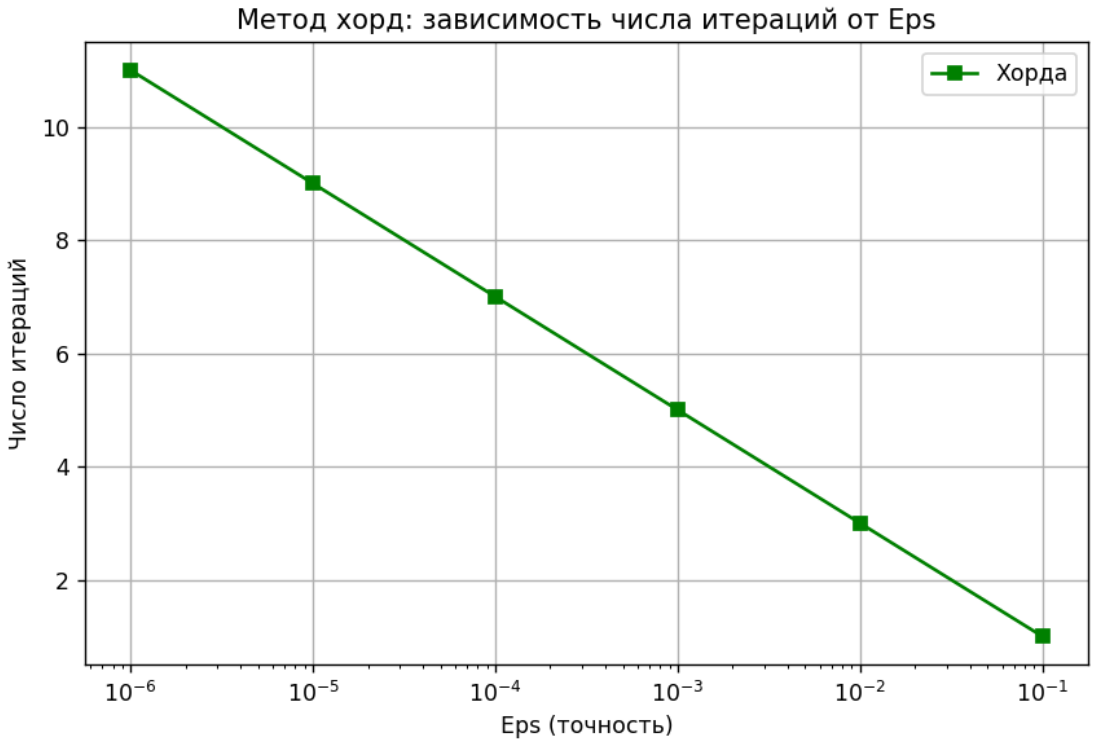

Рисунок 3 - Зависимости числа итераций от точности *Eps* в методе хорд

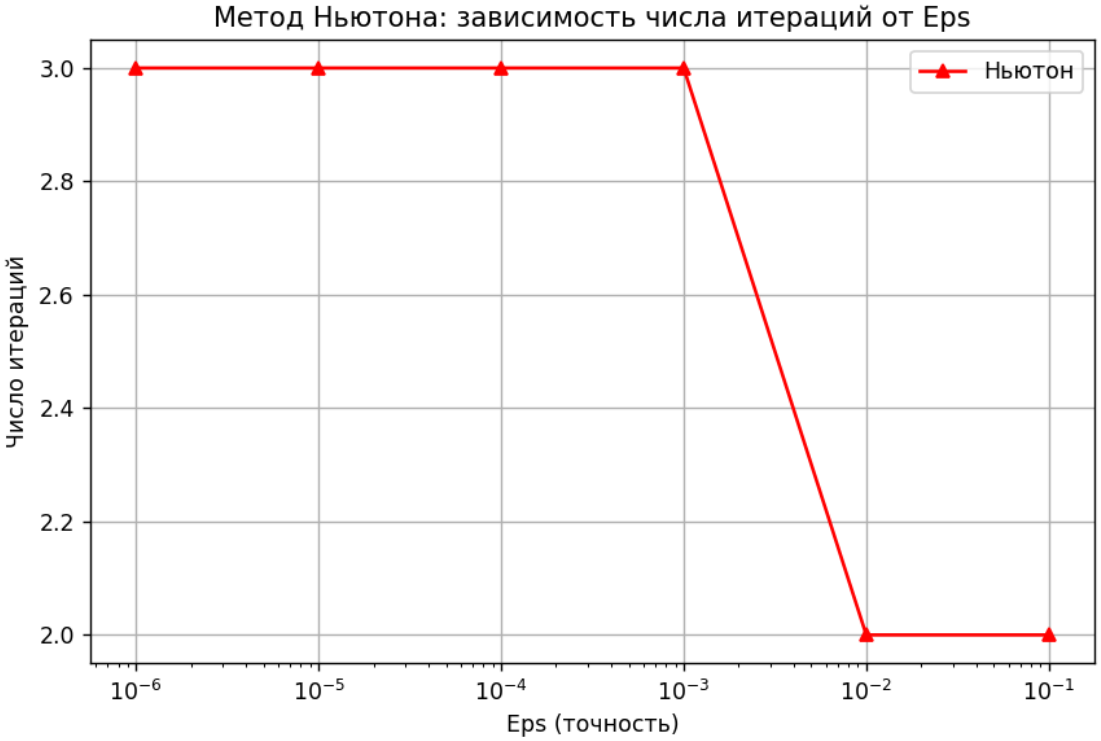

Рисунок 4 - Зависимости числа итераций от точности *Eps* в методе Ньютона

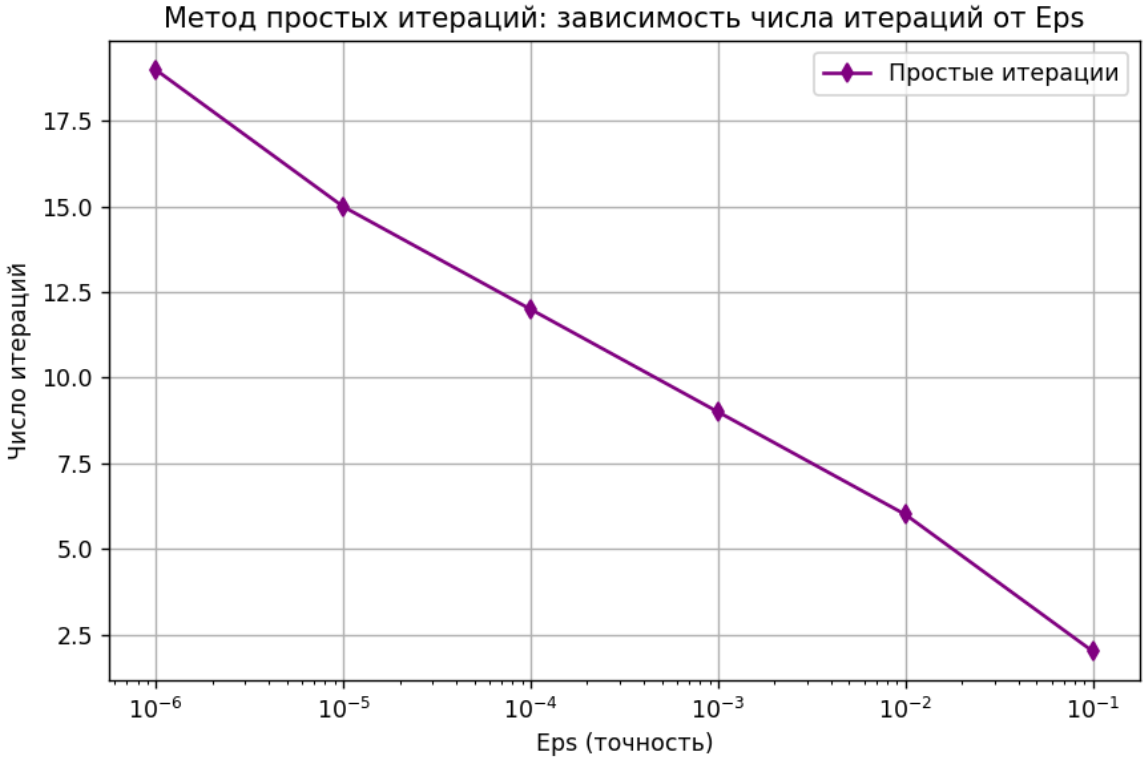

Рисунок 5 - Зависимости числа итераций от точности *Eps* в методе простых итераций

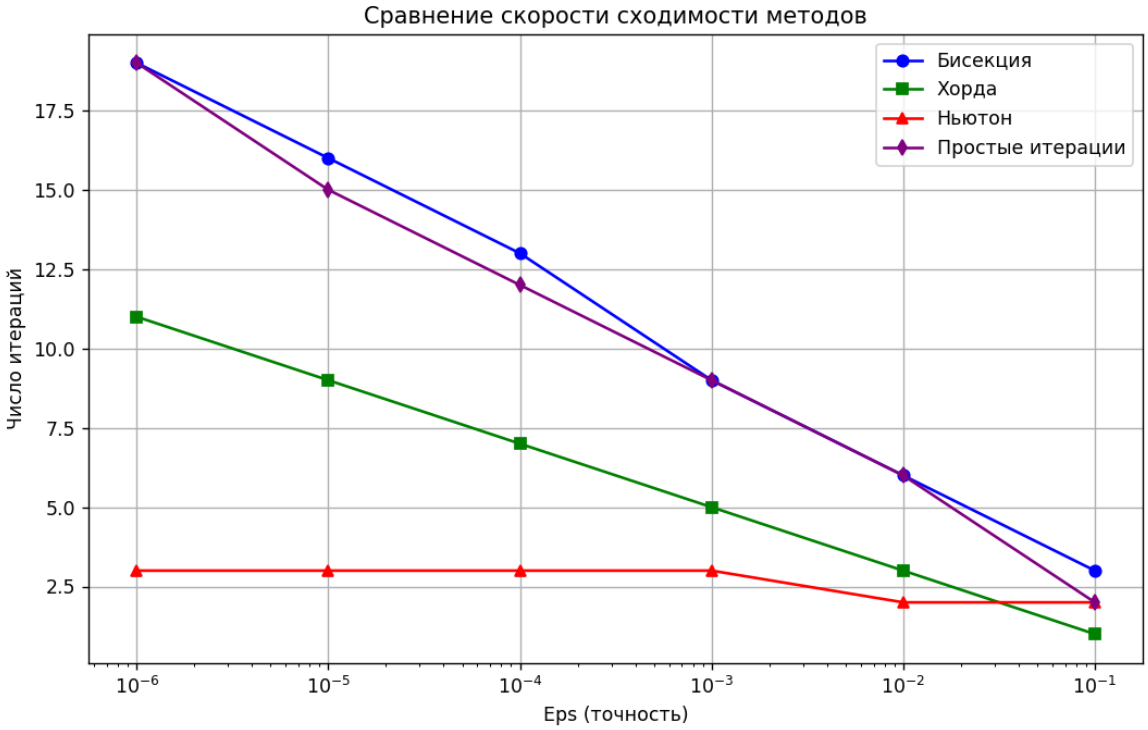

Рисунок 6 – Сравнение скорости сходимости методов

Моделирование ошибок исходных данных через глобальную переменную Delta показало, что даже небольшие погрешности влияют на результаты работы методов.

Метод бисекции: метод показал хорошую устойчивость к ошибкам округления. Устойчивость обеспечивается простотой вычислений: метод использует только сложение и деление на 2, что приводит к усреднению ошибок, а не их усилению. Проверка знака функции также устойчива к малым погрешностям. Однако критерий остановки $(b - a) < 2\varepsilon$ является косвенным – малый интервал не всегда гарантирует точный корень при наличии ошибок округления.

Метод хорд: метод продемонстрировал наивысшую устойчивость. Критерий остановки $|f(x)| < \varepsilon$ напрямую проверяет близость к корню: если значение функции мало, то x близко к корню независимо от накопленных ошибок. Метод обладает самокоррекцией — даже если текущее приближение имеет ошибку, следующая итерация исправляет траекторию к корню.

Метод Ньютона: метод оказался чувствительным к ошибкам. Деление на производную в формуле $x_{n+1} = x_n - \frac{f_n}{f_n'}$ усиливает ошибки: малая производная превращает небольшую ошибку в *f(x)* в большую ошибку приращения. Вычисление производной добавляет дополнительные погрешности. Улучшенный критерий частично компенсирует ошибки досрочной остановкой, но не предотвращает смещение корня. При малых *Delta* метод показывает высокую точность и быструю сходимость.

Метод простых итераций: метод показал наименьшую устойчивость. Ошибки накапливаются через большое число итераций. На каждой итерации ошибка в $\varphi(x)$ переносится на следующее приближение с коэффициентом $q \approx 0.54$.

Графики чувствительности методов к ошибкам см. на рисунках 7, 8, 9, 10 и 11.

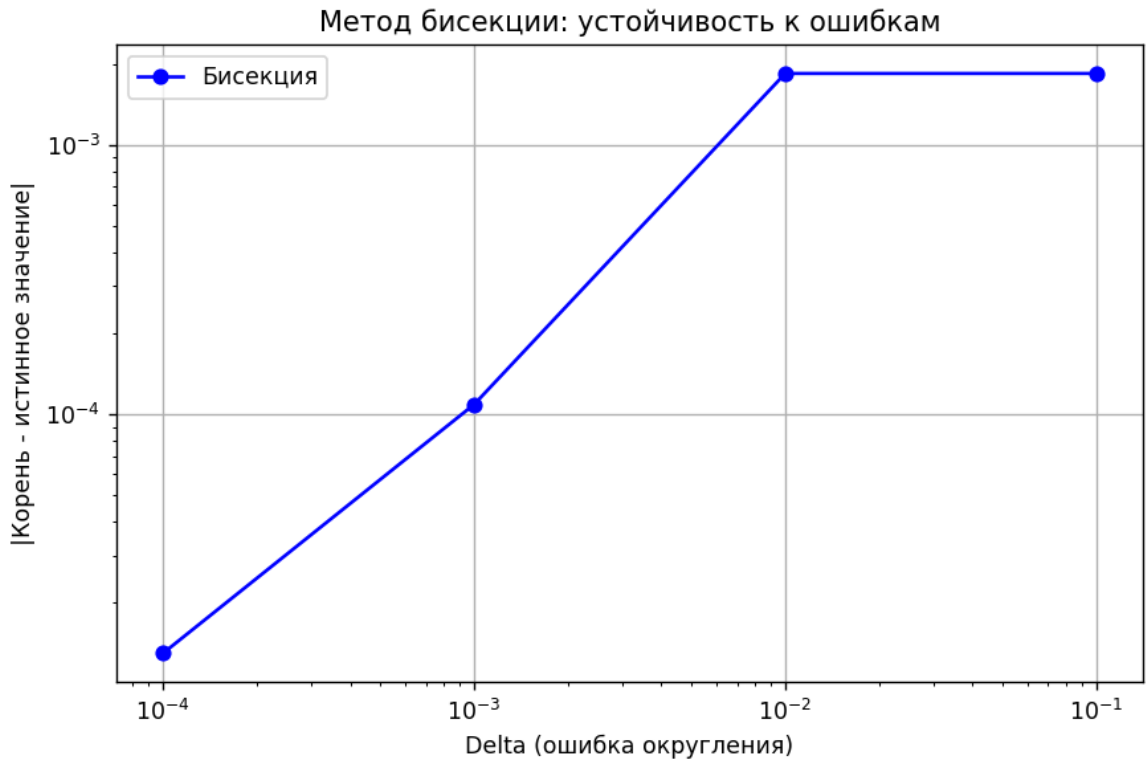

Рисунок 7 – Чувствительность к ошибкам метода бисекции

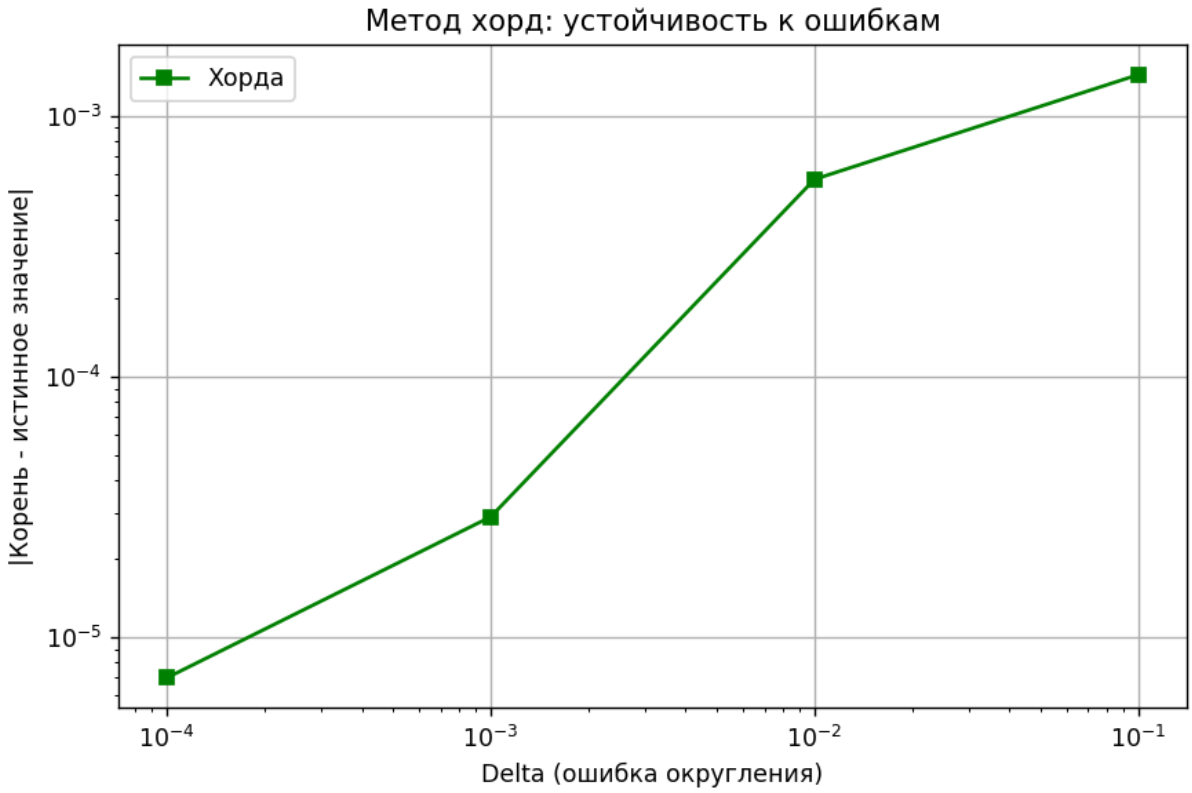

Рисунок 8 – Чувствительность к ошибкам метода хорд

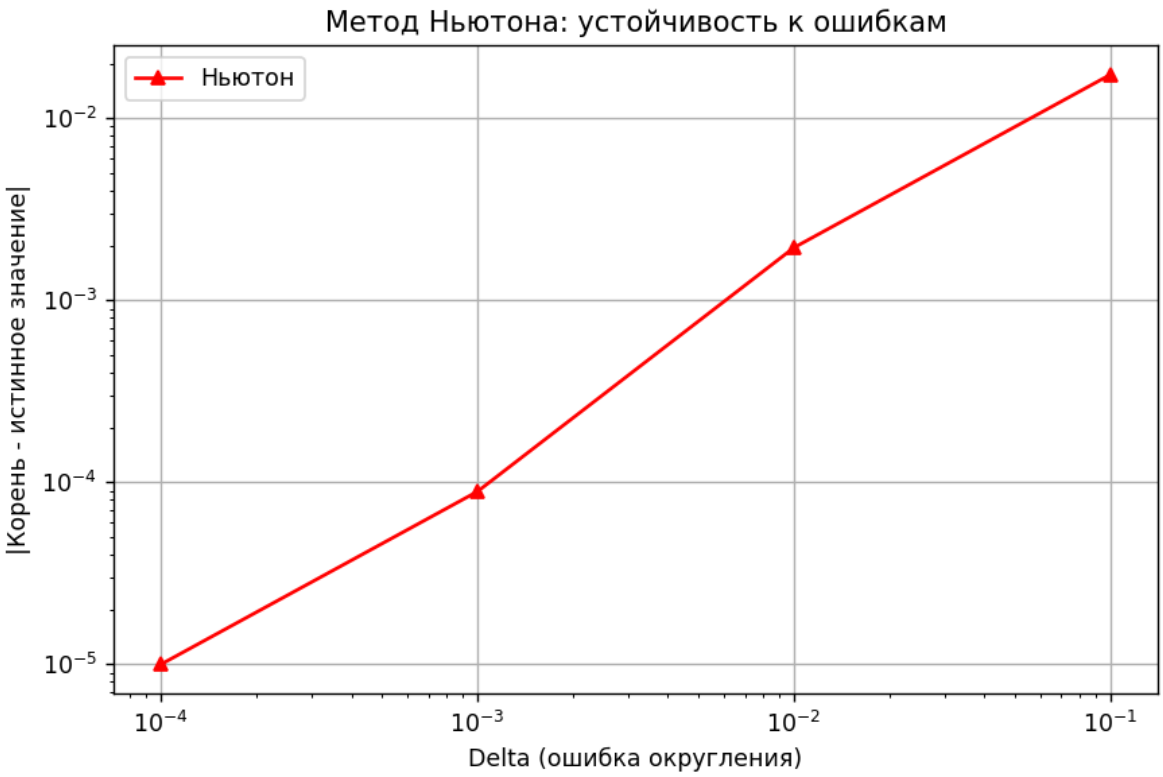

Рисунок 9 – Чувствительность к ошибкам метода Ньютона

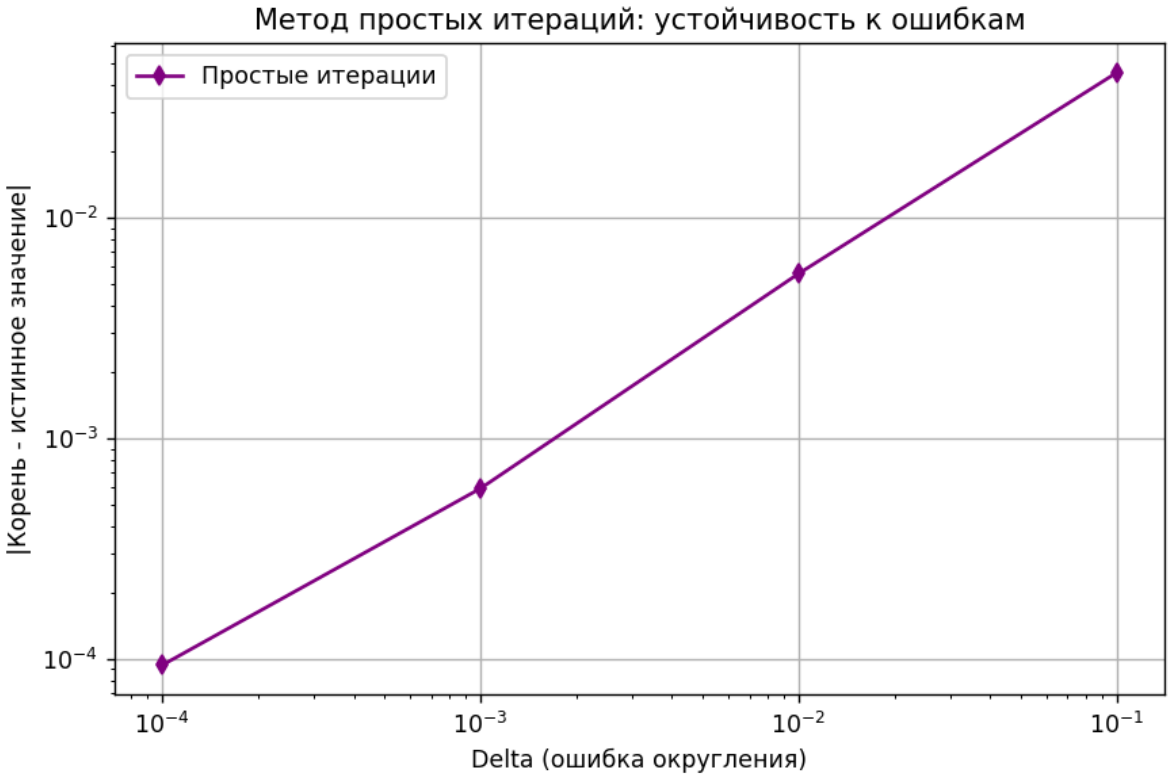

Рисунок 10 – Чувствительность к ошибкам метода простых итераций

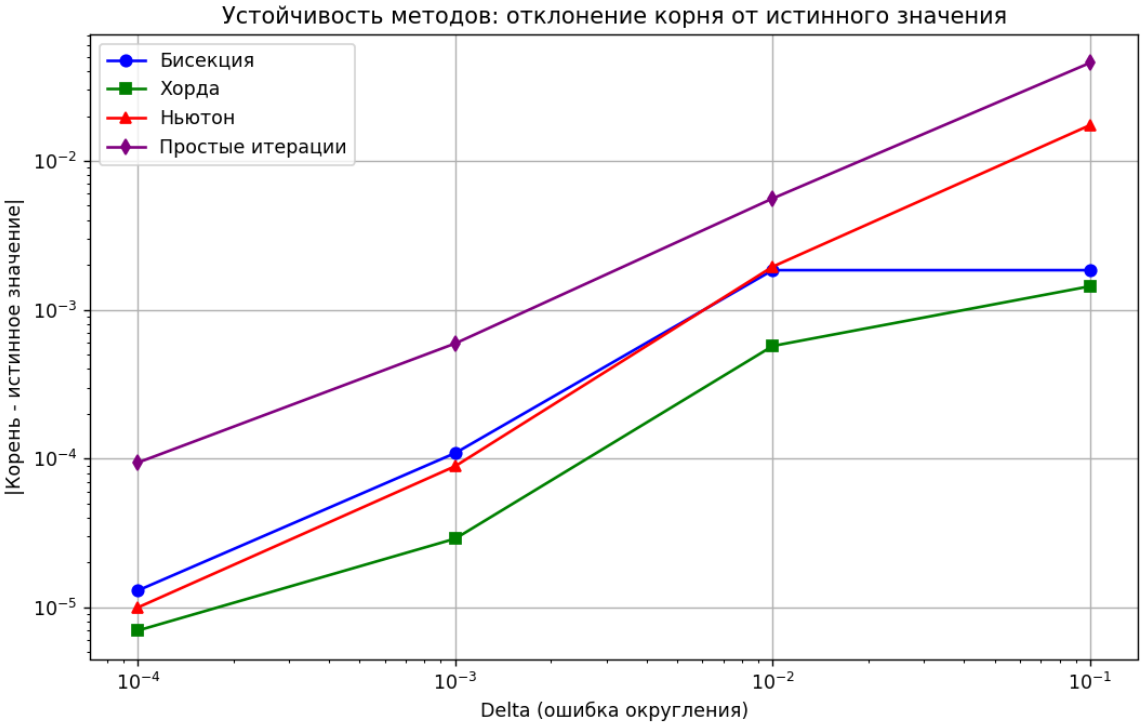

Рисунок 11 – Сравнение чувствительности к ошибкам методов

Вывод.

В ходе выполнения лабораторной работы были изучены и реализованы методы численного поиска корня нелинейного уравнения: метод бисекции, метод хорд, метод Ньютона и метод простых итераций. Были подробно рассмотрены принципы работы каждого метода, условия сходимости, а также особенности их скорости сходимости и устойчивости к ошибкам в исходных данных. Метод бисекции показал гарантированную сходимость и хорошую устойчивость к ошибкам благодаря простоте вычислений и усреднению погрешностей, но с относительно медленной линейной скоростью. Метод хорд продемонстрировал наилучшее сочетание характеристик: суперлинейную сходимость и наивысшую устойчивость к ошибкам среди всех методов благодаря критерию остановки, который напрямую проверяет близость к корню. Метод Ньютона обеспечил максимальную скорость сходимости за счет квадратичной сходимости, однако оказался чувствительным к ошибкам округления из-за деления на производную, усиливающего погрешности. Метод простых итераций с параметром релаксации $\lambda = 0.2$ показал линейную сходимость и наименьшую устойчивость к ошибкам вследствие накопления погрешностей через большое число итераций. Реализация методов с моделированием ошибок округления (параметр *Delta*) позволила на практике оценить влияние вычислительных погрешностей на точность результатов и подтвердить теоретические оценки скорости сходимости. Экспериментальные данные показали, что выбор метода зависит от требований задачи: для максимальной скорости при точных данных оптимален метод Ньютона, для баланса скорости и надежности — метод хорд, для гарантированной сходимости — метод бисекции.